

Voice Control AprilTag ID Sorting

1. Function Description

Voice commands are issued to sort AprilTag IDs. The robotic arm will grasp the AprilTag block corresponding to the ID in the command based on the instruction received.

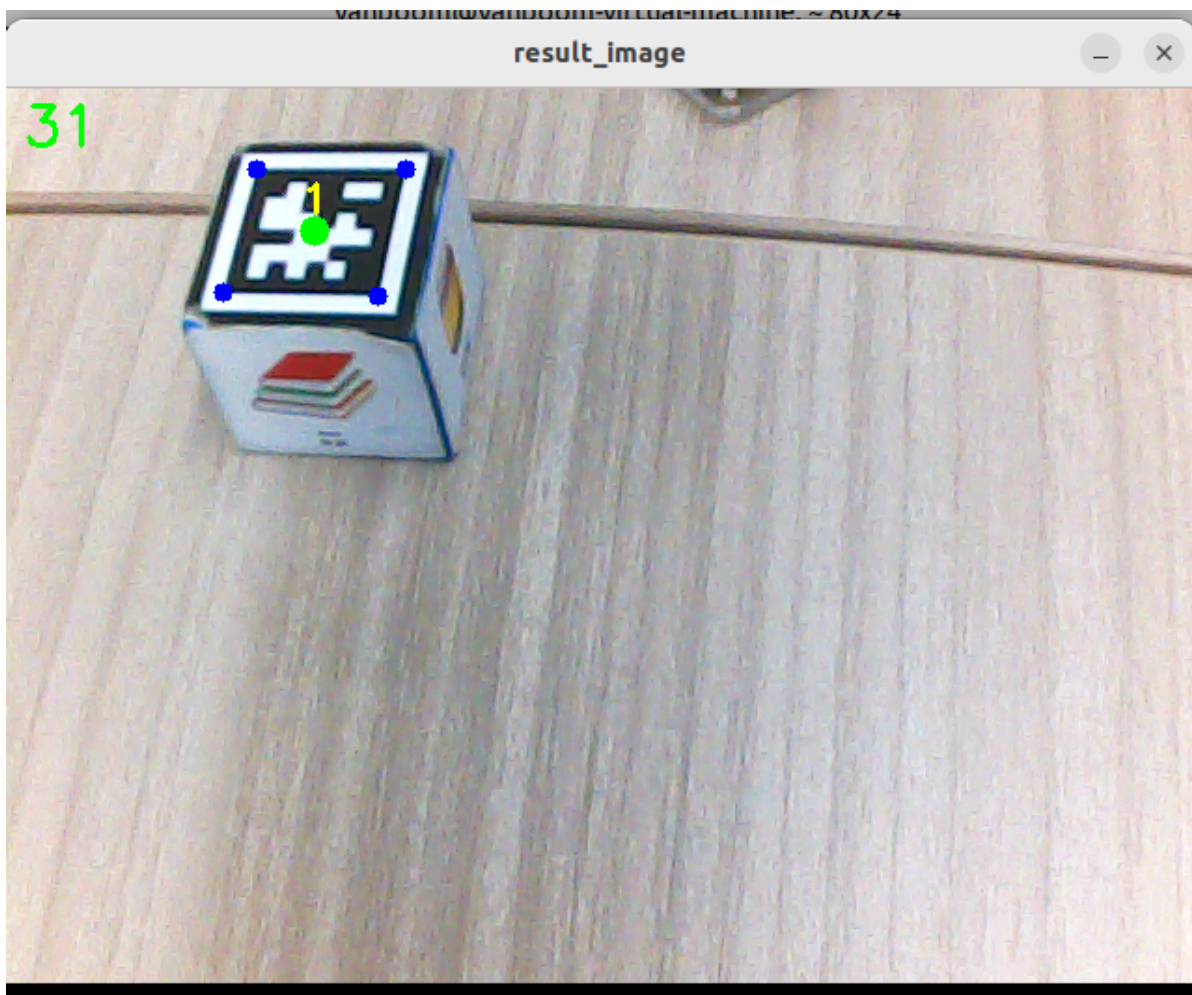
2. Startup and Operation

2.1 Startup

```
# Start robotic arm + camera inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start voice recognition program
ros2 launch yahboom_speech speech.launch.py
# Start robotic arm control program
ros2 run dofbot_voice_ctrl grasp_VC
# Start AprilTag recognition program
ros2 run dofbot_voice_ctrl apriltag_detect
```

2.2 Operation Steps

After all programs are running, place the four AprilTags with different IDs in the camera field of view. Say "Hi, yahboom" to the voice recognition module, and the speaker will announce "I am here". Then say "Sort AprilTag No.1"/"Sort AprilTag No.2"/"Sort AprilTag No.3"/"Sort AprilTag No.4" to the voice module. The robotic arm will lower its gripper to grasp the AprilTag block with the target ID, and the voice module will announce "OK". After grasping the AprilTag block, it will be placed at the designated location. Finally, the robotic arm returns to its initial pose, and the voice announcement module will announce "Placement complete".



3. Core Code Analysis

3.1 Voice Recognition Node speech_recognize_node

Source code path:

/home/yahboom/dofbot_ws/src/yahboom_speech/yahboom_speech/speech_recognize.py

```
import time
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int8
import pandas as pd
# Import voice recognition and announcement library
# 导入语音识别播报库
from Speech_Lib import Speech
# Create voice recognition and announcement object
# 创建语音识别播报的对象
mySpeech = Speech()
# Create voice recognition result publisher
# 创建语音识别结果发布者
self.pub_res = self.create_publisher(Int8, 'voice_result', 1)
# Call library function to get recognition result
# 调用库中的函数，得到识别结果
result = mySpeech.speech_read()
# Publish voice recognition result topic
# 发布语音识别结果话题
self.voice_command.data = result
self.pub_res.publish(self.voice_command)
```

3.2 Voice Announcement Node voice_player_node

Source code path:

/home/yahboom/dofbot_ws/src/yahboom_speech/yahboom_speech/voice_player.py

This section mainly describes how to create a subscriber for voice announcements and how to call the library function to announce voice content.

```
# Import voice recognition and announcement library
# 导入语音识别播报库
from Speech_Lib import Speech
# Create voice recognition and announcement object
# 创建语音识别播报的对象
mySpeech = Speech()

# Create subscriber for voice announcement result
# 创建订阅语音播报结果订阅者
self.sub_playID =
self.create_subscription(Int8, 'player_id', self.playercallback, 1000)

# Callback function
# 回调函数
def playercallback(self, msg):
    # Extract voice announcement topic data
    # 提取语音播报话题数据
    ID_value = msg.data
    print("ID_value: ", ID_value)
    if ID_value != 0:
        play_id = ID_value
        print("play_id = ", play_id)
        # Call library's void_write to play audio file
        # 调用库中的void_write来播放音频文件
        mySpeech.void_write(int(play_id))
```

3.3 AprilTag Recognition Node apriltag_detect.py

Source code path:

/home/yahboom/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/AprilTag/apriltag_detect.py

This section mainly describes how to subscribe to, process voice recognition result topics, and publish voice announcement topics.

```
# Create subscriber for voice recognition result topic
# 创建订阅语音识别结果话题的订阅者
self.sub_voice =
self.create_subscription(Int8, "voice_result", self.getVoiceResultCallback, 1)
# Create publisher for voice announcement topic
# 创建发布语音播报话题的发布者
self.pub_playID = self.create_publisher(Int8, "player_id", 1)

# Callback function
# 回调函数
def getVoiceResultCallback(self, msg):
    # When the voice recognition result topic data is 95, 96, 97, 98, it
    # represents the commands for sorting AprilTag No.1/2/3/4 respectively
```

```

# 当语音识别结果话题的数据为95、96、97、98时，分别表示分拣一/二/三/四号机器码的指令
if msg.data == 95 or msg.data == 96 or msg.data == 97 or msg.data == 98:
    # Change self.pubPos_flag to indicate that the AprilTag position
    information topic can be published
    # 改变self.pubPos_flag，标志着可以发布机器码位置信息话题
    self.pubPos_flag = True
    play_id = Int8()
    # Voice announcement result 45, playing audio "OK", publish this topic
    information to the voice announcement node
    # 语音播报结果45，播放的音频为"好的"，发布该话题信息到语音播报节点
    play_id.data = 45
    self.pub_playID.publish(play_id)

```

3.4 Robotic Arm Grasping AprilTag Node apriltag_grasp_VC.py

Source code path:

/home/yahboom/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/AprilTag/apriltag_grasp_VC.py

```

# Define placement position based on the ID of the AprilTag to be grasped
# 根据待抓取机器码的id定义放置位置
def move(self):
    self.Arm.Arm_serial_servo_write(5, 90, 2000)
    time.sleep(2.0)
    self.Arm.Arm_serial_servo_write(6, 140, 2000)
    time.sleep(2.0)
    self.Arm.Arm_serial_servo_write(6, self.joint_6, 2000)
    time.sleep(2.0)
    self.Arm.Arm_serial_servo_write(2, 120, 2000)
    time.sleep(2.0)
    print("self.cur_tagId", self.cur_tagId)
    # Blue
    # 蓝色
    p_1 = [30, 70, 0, 54, 265, self.joint_6]
    # Green
    # 绿色
    p_2 = [150, 70, 7, 56, 265, self.joint_6]
    # Red
    # 红色
    p_3 = [50, 70, 7, 58, 265, self.joint_6]
    # Yellow
    # 黄色
    p_4 = [135, 70, 7, 54, 265, self.joint_6]
    if self.cur_tagId == 1:
        self.down_joint = p_1
    elif self.cur_tagId == 2:
        self.down_joint = p_2
    elif self.cur_tagId == 3:
        self.down_joint = p_3
    elif self.cur_tagId == 4:
        self.down_joint = p_4
    print("self.down_joint", self.down_joint)
    self.down_joint[5] = self.joint_6
    self.Arm.Arm_serial_servo_write6_array(self.down_joint, 2000)
    time.sleep(2.0)
    self.Arm.Arm_serial_servo_write(6, 90, 2000)
    time.sleep(2.0)

```

```
self.Arm.Arm_serial_servo_write(2, 90, 2000)
time.sleep(2.0)
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
time.sleep(4)
# Publish grasp completion, reset grasp flag
# 发布抓取完成，重置抓取标志位
self.grasp_flag = True
grasp_done = Bool()
grasp_done.data = True
self.pubGraspStatus.publish(grasp_done)
```